南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Advanced Programming

Lab 10, Class(2): operator overloading

廖琪梅，王大兴，于仕琪，王薇

CC BY NC SA

Exercise 1

Modify the code(`rational.h`) so that the program runs as shown in the screenshot on the right and explain the output where each constructor is run.

```
// rational.h
#pragma once
#include <iostream>
class Rational{
private:
    staticint id;
    int numerator;
    int denominator;
public:
    Rational(int n=0, int d=1): numerator(n), denominator(d){
        std::cout<<"Construct_"<<id<<", n:"<<numerator<<" ,
d:"<<denominator<<std::endl;
    }

    int getN() const { return numerator; }
    int getD() const { return denominator; }
    friend std::ostream & operator<<(std::ostream& os, const Rational&rhs){
        os<<rhs.numerator<<"/"<<rhs.denominator;
        returnos;
    }
};
int Rational::id=0;
const Rational operator*(constRational&lhs, const Rational&rhs){
        return Rational(lhs.getN()*rhs.getN(), lhs.getD()*rhs.getD() );
}
```

```
#include <iostream>
#include "rational.h"
using namespace std;
int main(){
    Rational a=10;
    Rational b(1,2);
    Rational c=a*b;
    cout<<"c = "<<c<<endl;
    Rational d=2*a;
    cout<<"d = "<<d<<endl;
    Rational e=b*3;
    cout<<"e = "<<e<<endl;
    Rational f=2*3;
    cout<<"f = "<<f<<endl;
    return0;
}
```

```
Construct_1, n:10 , d:1
Construct_2, n:1 , d:2
Construct_3, n:10 , d:2
c = 10/2
Construct_4, n:2 , d:1
Construct_5, n:20 , d:1
d = 20/1
Construct_6, n:3 , d:1
Construct_7, n:3 , d:2
e = 3/2
Construct_8, n:6 , d:1
f = 6/1
```

CC BY NC SA

Exercise 2

Please modify the code(`rational.h`) in Exercise 1 to the program runs as shown in the screenshot. Explain what modifications have been made and why.

```
// rational.h
#pragma once
#include <iostream>
class Rational{
private:
    staticint id;
    int numerator;
    int denominator;
public:
    Rational(int n=0, int d=1): numerator(n), denominator(d){
        std::cout<<"Construct_"<<id<<", n:"<<numerator<<" ,
d:"<<denominator<<std::endl;
    }

    int getN() const { return numerator; }
    int getD() const { return denominator; }
    friend std::ostream & operator<<(std::ostream& os, const Rational&rhs){
        os<<rhs.numerator<<"/"<<rhs.denominator;
        returnos;
    }
};
int Rational::id=0;
const Rational operator*(constRational&lhs, const Rational&rhs){
        return Rational(lhs.getN()*rhs.getN(), lhs.getD()*rhs.getD() );
}
```

```
#include <iostream>
#include "rational.h"
using namespace std;
int main(){
    Rational a=10;
    Rational b(1,2);
    Rational c=a*b;
    cout<<"c = "<<c<<endl;
    Rational d=2*a;
    cout<<"d = "<<d<<endl;
    Rational e=b*3;
    cout<<"e = "<<e<<endl;
    Rational f=2*3;
    cout<<"f = "<<f<<endl;
    return0;
}
```

```
Construct_1, n:10 , d:1
Construct_2, n:1 , d:2
Construct_3, n:10 , d:2
c = 10/2
Construct_4, n:20 , d:1
d = 20/1
Construct_5, n:3 , d:2
e = 3/2
Construct_6, n:6 , d:1
f = 6/1
```

CC BY NC SA

Exercise 3

Continue to improve the Complex class and add more operations for it, such as: **-, *, ~, ==, !=** etc. Make the following program run correctly.

```cpp
#include <iostream>
#include "complex.h"
using namespace std;

int main()
{
    Complex a(3, 4);
    Complex b (2,6);

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "~b = " << ~b << endl;
    cout << "a + b = " << a+b << endl;
    cout << "a - b = " << a-b << endl;
    cout << "a - 2 = " << a-2 << endl;
    cout << "a * b = " << a*b << endl;
    cout << "2 * a = " << 2*a << endl;
    cout << "============================" << endl;

    Complex c = b;
    cout << "c = " << c << endl;
    cout << "b == c? " << boolalpha << (b==c) << endl;
    cout << "b != c? " << (b!=c) << endl;
    cout << "a == b? " << (a==b) << endl;
    cout << "============================" << endl;
    Complex d;
    cout << "Enter a complex number(real part and imaginary part):";
    cin >> d;
    cout << "Before assignment, d = " << d << endl;
    d = c;
    cout << "After assignment, d = " << d << endl;

    return 0;
}
```

Note that you have to overload the **<< and >> operators**. Use the references to objects and const whenever warranted. The sample should run as follows:

```
a = 3+4i
b = 2+6i
~b = 2-6i
a + b = 5-2i
a - b = 1+10i
a - 2 = 1+4i
a * b = 30-10i
2 * a = 6+8i
============================
c = 2-6i
b == c? true
b != c? false
a == b? false
============================
Enter a complex number(real part and imaginary part):4 6
Before assignment, d = 4+6i
After assignment, d = 2-6i
```

CC BY NC SA